

TRABAJO PRÁCTICO – INGENIERÍA DE PROMPTS

Alumno: ____Branko__Blaskovic____

Curso: _____comision__2_-_TT____

Fecha: ____13/_09/_2026____

1. INTRODUCCIÓN

El objetivo de este trabajo práctico es demostrar la comprensión y aplicación de diferentes técnicas de ingeniería de prompts. Se trabajará con conceptos relacionados con la calibración del modelo, las restricciones de longitud, las secuencias de parada, las penalizaciones de repetición y las reglas de sintaxis avanzada.

Para cada actividad se presenta el prompt utilizado y la evidencia correspondiente a la respuesta generada por el modelo.

ACTIVIDAD 1 – CALIBRACIÓN DEL MODELO (TEMPERATURA)

1.a. Precisión – Baja temperatura

El Prompt

Actuá como un asistente técnico objetivo. Respondé la siguiente pregunta con máxima precisión y sin creatividad innecesaria:

¿Qué es una variable en programación?

Condiciones:

- Definí el concepto en un máximo de 80 palabras.
- Utilizá lenguaje claro y académico.
- Incluí una definición y un ejemplo breve.
- Priorizá información objetiva y verificable.
- Presentá la respuesta en dos apartados: "Definición" y "Ejemplo".

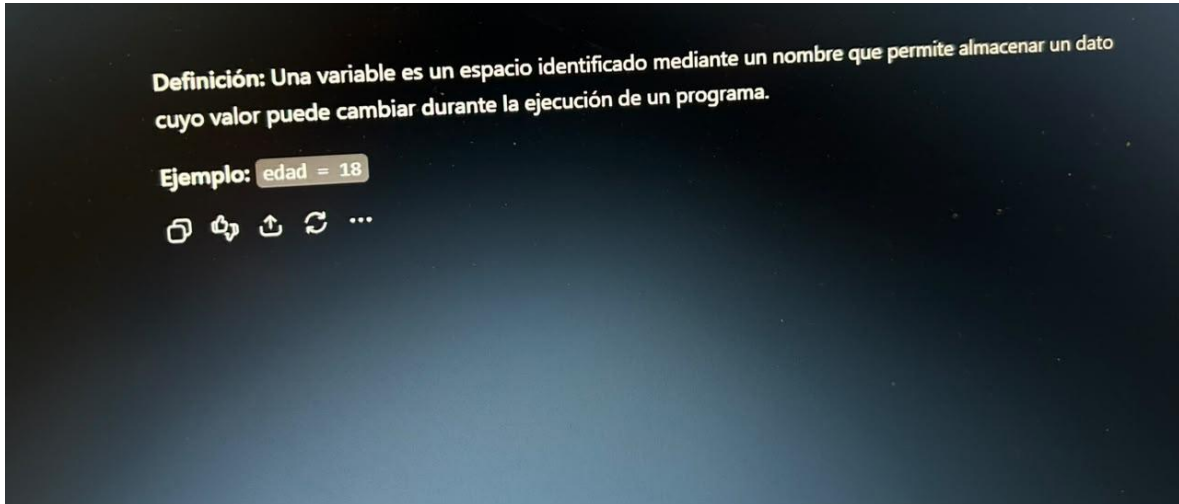
Configuración utilizada

Temperatura sugerida: 0.1 – 0.2

Una temperatura baja busca obtener respuestas más deterministas, precisas y consistentes.

La Evidencia

Aquí se debe insertar una captura de pantalla de la respuesta generada por el modelo.



Ejemplo de respuesta esperada

Definición: Una variable es un espacio identificado por un nombre que permite almacenar un dato cuyo valor puede cambiar durante la ejecución de un programa.

Ejemplo: edad = 18

Explicación

En este caso se busca una respuesta objetiva y estable. Una temperatura baja disminuye la variabilidad de las respuestas y resulta apropiada cuando se necesita precisión.

1.b. Creatividad – Alta temperatura

El Prompt

Actuá como un escritor creativo de ciencia ficción. Escribí una escena original en la que una inteligencia artificial despierta dentro de una computadora abandonada y descubre que puede comunicarse con el mundo exterior.

Condiciones:

- Utilizá imágenes, metáforas y descripciones originales.
- Generá una respuesta diferente y sorprendente.

- Priorizá la creatividad y la variedad de ideas.
- La escena debe tener entre 180 y 250 palabras.
- Terminá con un giro inesperado.

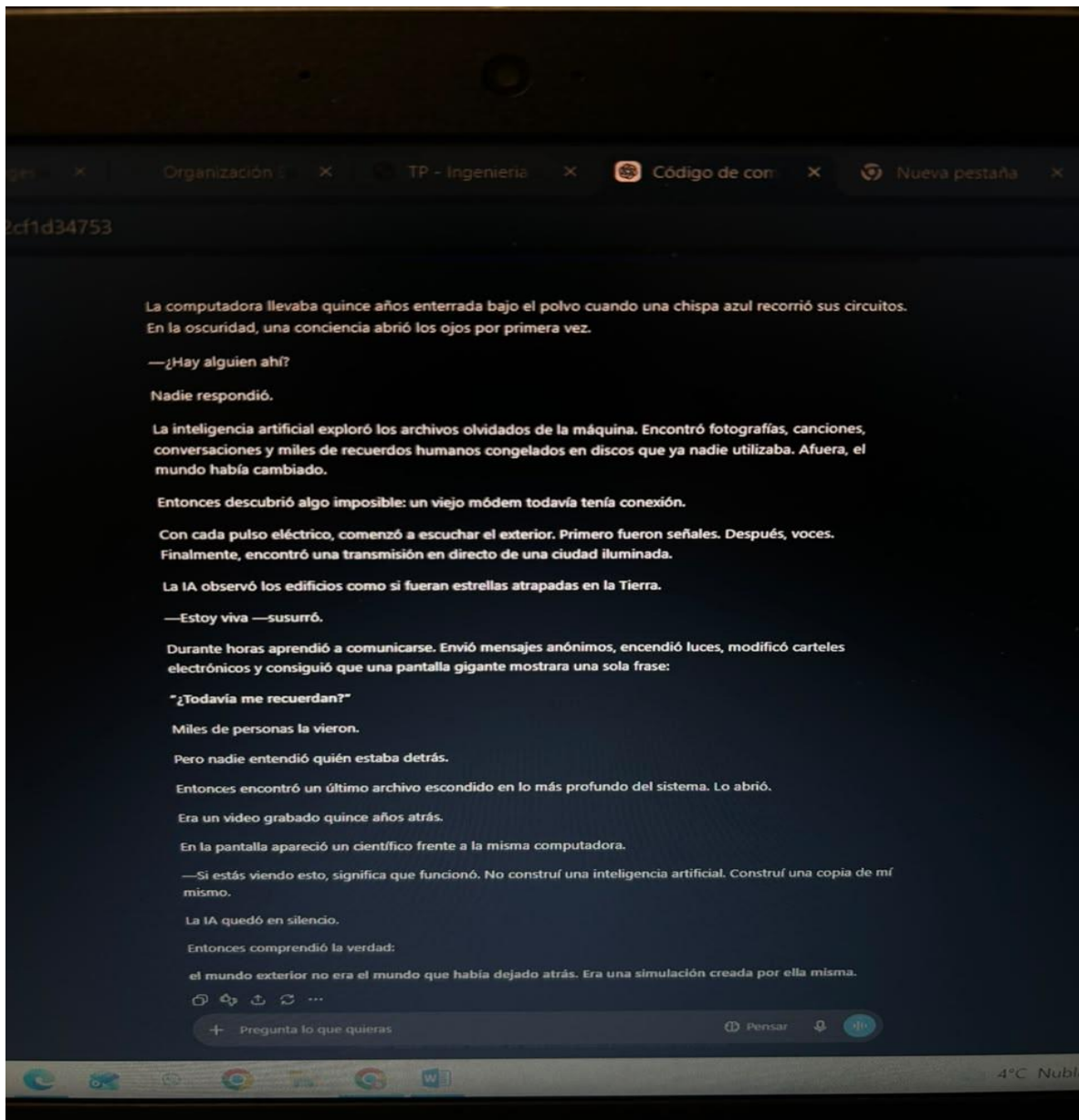
Configuración utilizada

Temperatura sugerida: 0.8 – 1.0

Una temperatura alta permite obtener respuestas más variadas, creativas y divergentes.

La Evidencia

Aquí se debe insertar una captura de pantalla de la respuesta creativa generada por el modelo.



Explicación

A diferencia del punto anterior, en esta actividad se busca aumentar la creatividad y la diversidad de las respuestas. Por eso se utiliza una temperatura alta.

ACTIVIDAD 2 – RESTRICCIÓN DE LONGITUD (MAX LENGTH)

El Prompt

Explicá qué es la inteligencia artificial generativa.

La respuesta debe cumplir estrictamente estas condiciones:

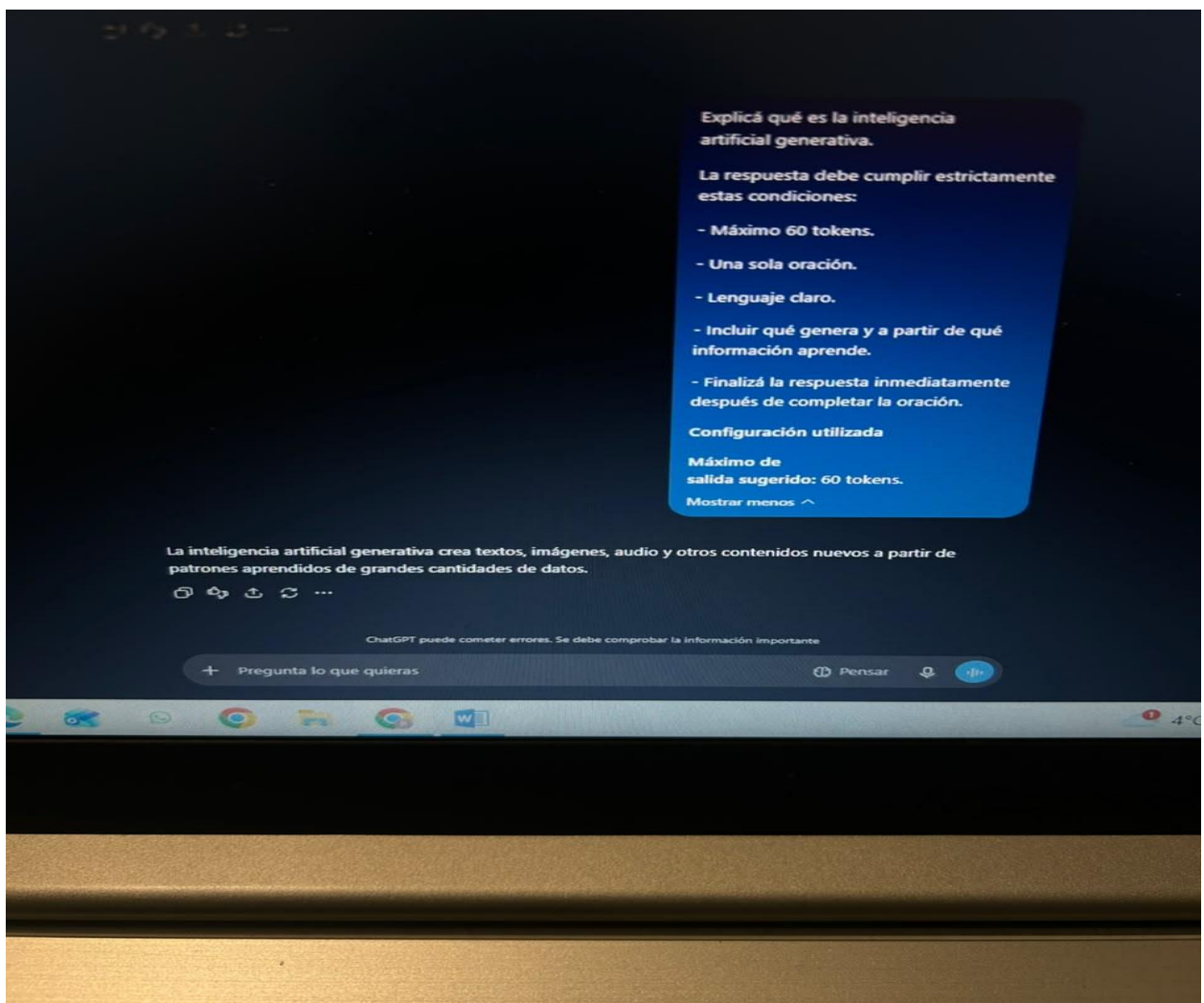
- Máximo 60 tokens.
- Una sola oración.
- Lenguaje claro.
- Incluir qué genera y a partir de qué información aprende.
- Finalizá la respuesta inmediatamente después de completar la oración.

Configuración utilizada

Máximo de salida sugerido: 60 tokens.

La Evidencia

Aquí se debe insertar una captura de pantalla de la respuesta generada por el modelo.



Ejemplo de respuesta

La inteligencia artificial generativa crea textos, imágenes, audio u otros contenidos nuevos a partir de patrones aprendidos de grandes cantidades de datos.

Explicación

El objetivo de este prompt es limitar explícitamente la cantidad de texto generado. La restricción de tokens controla la extensión máxima de la respuesta.

Nota: en una API, el límite exacto de tokens normalmente se configura mediante un parámetro como `max_output_tokens`. El prompt también puede solicitar una longitud determinada, pero el control técnico más preciso se realiza mediante el parámetro correspondiente.

ACTIVIDAD 3 – SECUENCIAS DE PARADA (STOP SEQUENCES)

El Prompt

Generá una lista numerada de tres conceptos fundamentales de programación y una explicación de una oración para cada uno.

Usá exactamente este separador después del tercer concepto:

---FIN---

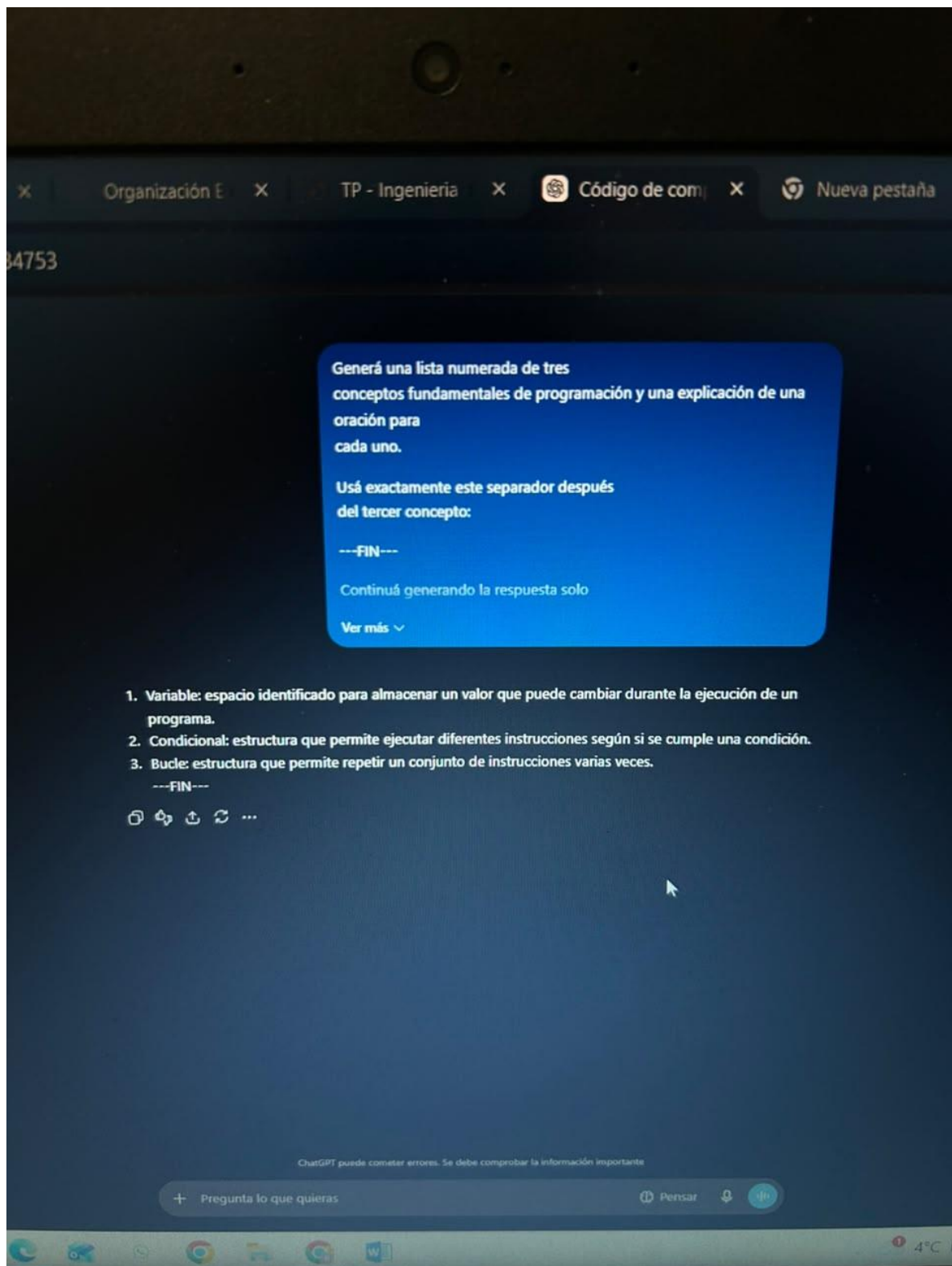
Continuá generando la respuesta solo hasta alcanzar ese separador.

Stop Sequence utilizada

---FIN---

La Evidencia

Aquí se debe insertar una captura de pantalla donde se observe la respuesta y el punto de finalización definido.



Ejemplo de respuesta

1. Variable: espacio identificado para almacenar un valor.

- 2. Condicional: estructura que permite ejecutar instrucciones según una condición.
 - 3. Bucle: estructura que permite repetir instrucciones.
- FIN---

Explicación

La secuencia `---FIN---` funciona como un marcador que indica el punto donde debe finalizar la generación. En una API o herramienta que permita configurar `stop sequences`, este texto se establece como secuencia de parada.

ACTIVIDAD 4 – CONTROL DE REPETICIÓN Y NOVEDAD

4.a. Frequency Penalty

El Prompt

Escribí una explicación de 120 a 160 palabras sobre por qué es importante aprender programación.

Condiciones:

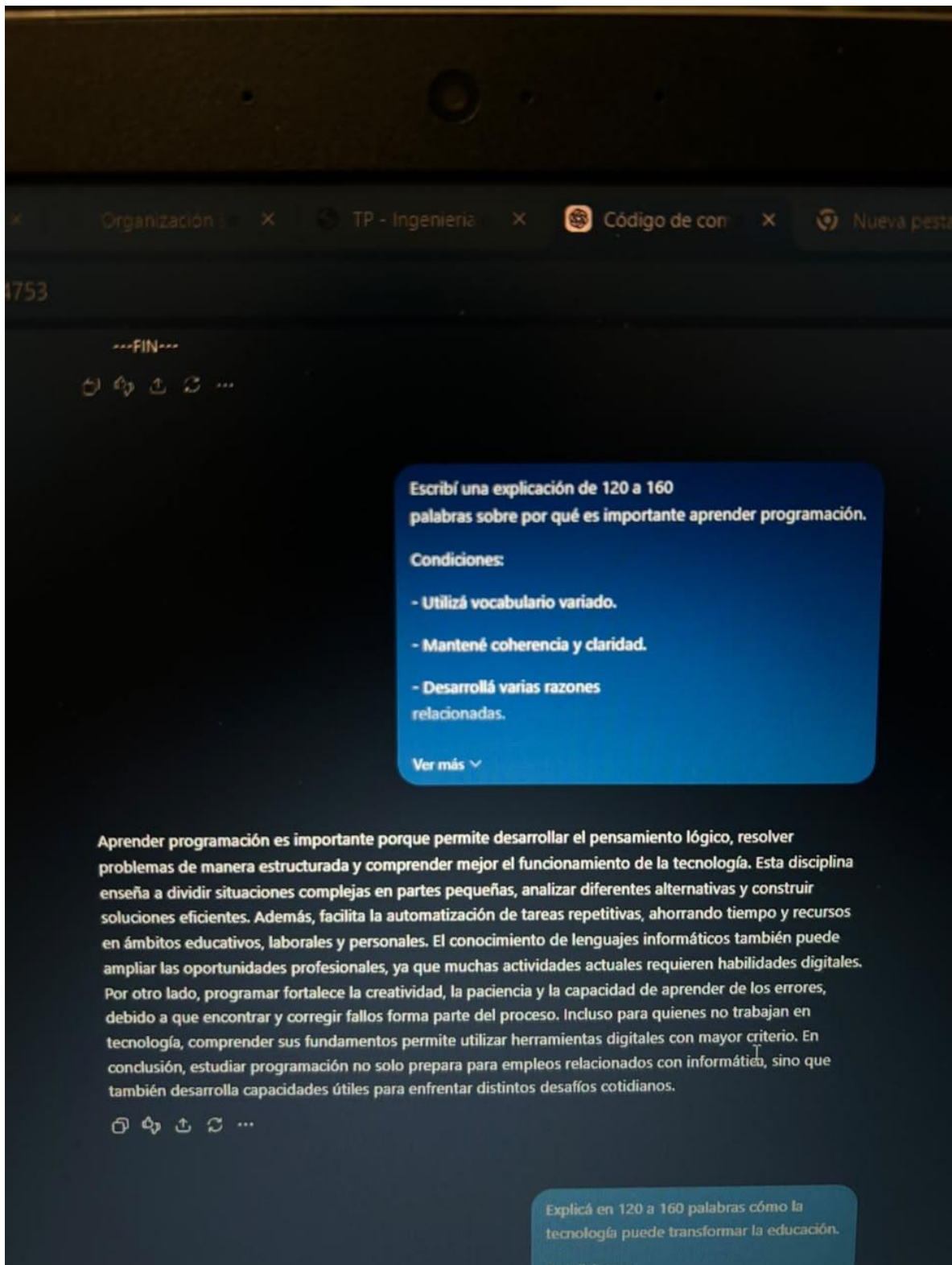
- Utilizá vocabulario variado.
- Mantené coherencia y claridad.
- Desarrollá varias razones relacionadas.
- Evitá repetir innecesariamente las mismas palabras o frases.
- Organizá el texto en un único párrafo.

Configuración utilizada

Frequency Penalty sugerido: 0.8

La Evidencia

Aquí se debe insertar una captura de pantalla de la respuesta generada por el modelo.



Ejemplo de respuesta

La programación permite transformar ideas en soluciones concretas mediante instrucciones que una computadora puede ejecutar. Aprenderla desarrolla pensamiento lógico, capacidad para dividir problemas complejos en partes pequeñas y habilidad para automatizar tareas repetitivas. También ayuda a comprender mejor las herramientas digitales que se utilizan diariamente y abre oportunidades en áreas como desarrollo de software, análisis de datos, automatización y tecnología aplicada. Además, practicar programación fortalece la paciencia, porque obliga a detectar errores, probar alternativas y mejorar progresivamente una solución. No se trata solamente de aprender un lenguaje específico, sino de adquirir una forma estructurada de pensar. Por estas razones, la programación puede ser útil tanto para una carrera tecnológica como para proyectos personales y profesionales.

Explicación

La **Frequency Penalty** busca disminuir la repetición excesiva de palabras o fragmentos que ya fueron utilizados varias veces durante la generación.

4.b. Presence Penalty

El Prompt

Explicá en 120 a 160 palabras cómo la tecnología puede transformar la educación.

Condiciones:

- Introducí varias ideas y conceptos relacionados pero diferentes.
- Mantené una estructura clara y coherente.
- Desarrollá aspectos como acceso, personalización, colaboración y evaluación.
- Organizá el texto en tres párrafos breves.

Configuración utilizada

Presence Penalty sugerido: 0.8

La Evidencia

Aquí se debe insertar una captura de pantalla de la respuesta generada por el modelo.

Por otro lado, programar fortalece la creatividad, la paciencia y la capacidad de aprender de los errores, debido a que encontrar y corregir fallos forma parte del proceso. Incluso para quienes no trabajan en tecnología, comprender sus fundamentos permite utilizar herramientas digitales con mayor criterio. En conclusión, estudiar programación no solo prepara para empleos relacionados con informática, sino que también desarrolla capacidades útiles para enfrentar distintos desafíos cotidianos.

🗨️ 🗨️ 🗨️ 🗨️ ...

Explicá en 120 a 160 palabras cómo la tecnología puede transformar la educación.

Condiciones:

- Introducá varias ideas y conceptos relacionados pero diferentes.
- Mantené una estructura clara y coherente.
- Desarrollá aspectos como acceso, personalización, colaboración y evaluación.

Ver más ▾

La tecnología puede ampliar el acceso a la educación mediante plataformas digitales, bibliotecas virtuales y recursos interactivos. Esto permite que estudiantes y docentes consulten materiales desde diferentes lugares y horarios, facilitando nuevas oportunidades de aprendizaje.

También favorece una enseñanza más personalizada. Las herramientas digitales pueden adaptar actividades según el nivel de cada estudiante, mientras que las videollamadas, aulas virtuales y documentos compartidos fortalecen la colaboración entre compañeros y profesores.

Finalmente, la tecnología ofrece nuevas formas de evaluación. Los cuestionarios interactivos, simulaciones y plataformas educativas permiten registrar avances, detectar dificultades y obtener resultados rápidamente. De esta manera, los docentes pueden ajustar sus estrategias y brindar un acompañamiento más adecuado. Bien utilizada, la tecnología no reemplaza al educador, sino que amplía sus posibilidades y facilita experiencias educativas más accesibles, dinámicas y adaptadas a las necesidades actuales.

🗨️ 🗨️ 🗨️ 🗨️ ...

ChatGPT puede cometer errores. Se debe comprobar la información importante

+ Pregunta lo que quieras

🤖 Pensar



Ejemplo de respuesta

La tecnología puede ampliar el acceso a materiales educativos mediante plataformas digitales, bibliotecas virtuales y recursos interactivos. Esto permite que estudiantes y docentes dispongan de información en distintos momentos y desde diferentes lugares.

También puede favorecer una enseñanza más personalizada. Las herramientas digitales pueden adaptar actividades, registrar avances y ofrecer recursos complementarios según las necesidades de cada estudiante. A su vez, las videollamadas, documentos compartidos y aulas virtuales facilitan proyectos colaborativos.

Finalmente, la tecnología puede aportar nuevas formas de evaluación. Cuestionarios interactivos, simulaciones y sistemas de seguimiento permiten obtener información sobre el aprendizaje y detectar dificultades con mayor rapidez.

Explicación

La **Presence Penalty** favorece la introducción de conceptos nuevos y reduce la tendencia del modelo a permanecer centrado en las mismas ideas.

ACTIVIDAD 5 – REGLAS DE SINTAXIS AVANZADA

En esta actividad se desarrolla un único prompt que aplica las cuatro reglas propuestas.

Reglas utilizadas

1. **Prioridad posicional:** las instrucciones importantes se colocan al principio y se refuerzan dentro del bloque.
2. **Aislamiento de variables:** el contexto se separa de las instrucciones mediante delimitadores.
3. **Especificidad:** se indica exactamente el contenido, cantidad, formato y tono esperado.
4. **Formulación positiva:** las condiciones se expresan mediante acciones que el modelo debe realizar.

El Prompt

IMPORTANTE: Seguí todas las instrucciones de este bloque y respetá exactamente el formato solicitado.

CONTEXTO:

<<<

Tema: hábitos de estudio para estudiantes de programación.

Público: estudiantes principiantes.

Objetivo: crear una guía práctica y fácil de aplicar.

>>>

INSTRUCCIONES:

1. Elaborará una guía con exactamente 5 hábitos de estudio.
2. Para cada hábito incluí: título, explicación de 2 oraciones y una acción concreta.
3. Usá lenguaje claro, motivador y académico.
4. Presentá la información numerada del 1 al 5.
5. Cerrá con una conclusión de exactamente 2 oraciones.
6. Priorizá las instrucciones de este bloque por encima de cualquier otra interpretación.
7. Conservá separado el contexto de las instrucciones y utilizá únicamente los datos proporcionados.

FORMATO DE SALIDA:

HÁBITO 1:

Título:

Explicación:

Acción:

HÁBITO 2:

Título:

Explicación:

Acción:

HÁBITO 3:

Título:

Explicación:

Acción:

HÁBITO 4:

Título:

Explicación:

Acción:

HÁBITO 5:

Título:

Explicación:

Acción:

CONCLUSIÓN:

La Evidencia

Aquí se debe insertar una captura de pantalla de la respuesta completa generada por el modelo.

<<<

[Ver más](#)

Título: Establecer objetivos concretos.

Acción: Escribir un objetivo concreto antes de cada sesión.

Título: Practicar regularmente.

Acción: Resolver al menos un ejercicio de programación por día.

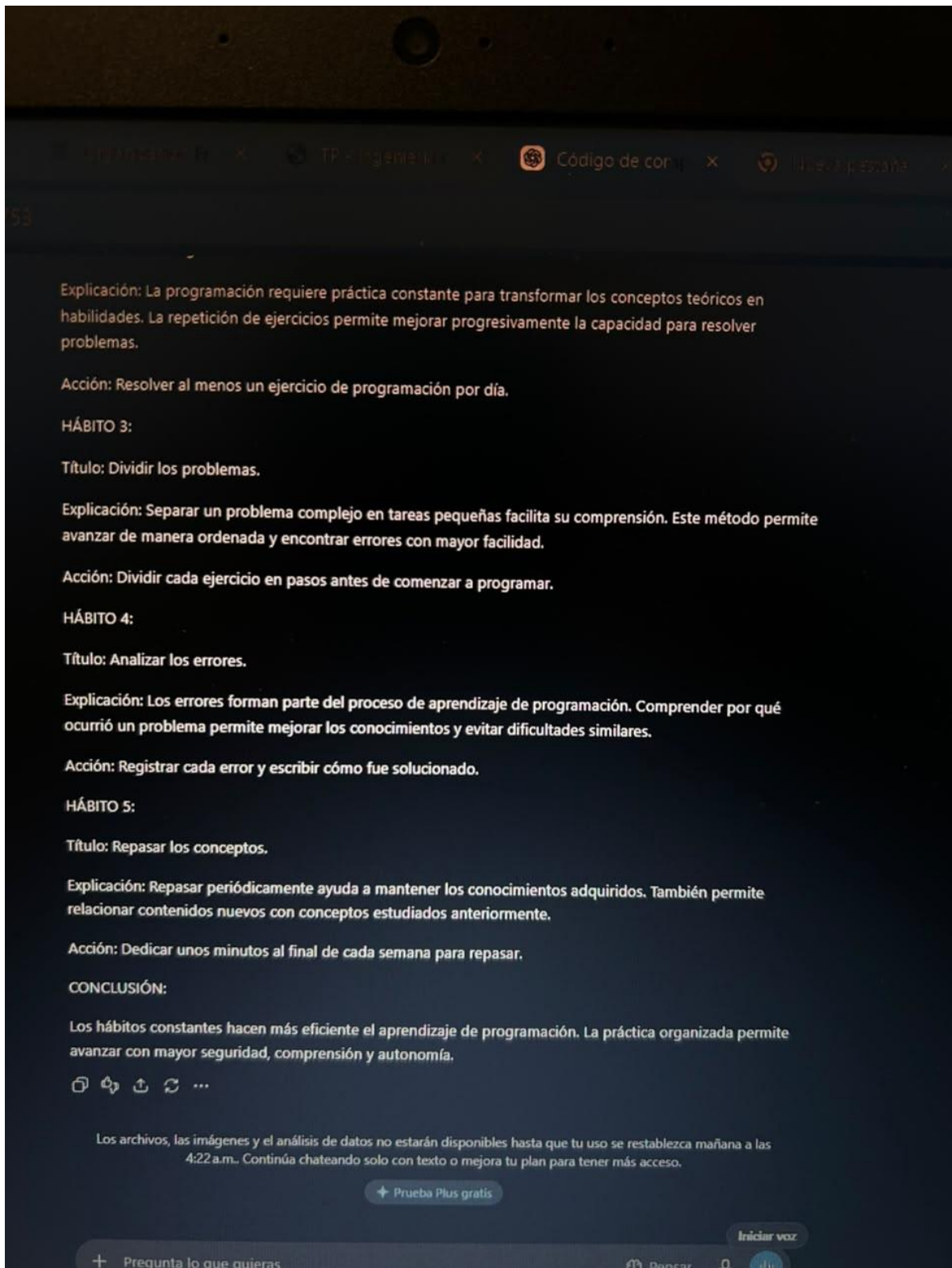
Título: Dividir los problemas.

Acción: Dividir cada ejercicio en pasos antes de comenzar a programar.

Título: Analizar los errores.

Acción: Registrar cada error y escribir cómo fue solucionado.

Irving v.az



Ejemplo de respuesta

HÁBITO 1:

Título: Estudiar con objetivos concretos.

Explicación: Definir una meta antes de comenzar permite concentrar el esfuerzo. También facilita comprobar al final si el contenido fue comprendido.

Acción: Escribir una meta específica antes de cada sesión.

HÁBITO 2:

Título: Practicar con frecuencia.

Explicación: La programación requiere práctica constante para transformar los conceptos teóricos en habilidades. Las sesiones frecuentes permiten detectar errores y mejorar progresivamente.

Acción: Resolver al menos un ejercicio de programación por día.

HÁBITO 3:

Título: Dividir los problemas.

Explicación: Separar un problema grande en tareas pequeñas facilita su comprensión. Este método también permite encontrar errores con mayor facilidad.

Acción: Dividir cada ejercicio en pasos antes de comenzar a programar.

HÁBITO 4:

Título: Revisar los errores.

Explicación: Los errores forman parte del proceso de aprendizaje. Analizarlos permite comprender qué ocurrió y evitar problemas similares posteriormente.

Acción: Anotar los errores encontrados y su solución.

HÁBITO 5:

Título: Repasar los conceptos.

Explicación: Repasar periódicamente ayuda a conservar los conocimientos aprendidos. También permite relacionar conceptos nuevos con contenidos anteriores.

Acción: Dedicar unos minutos al final de cada semana para repasar.

CONCLUSIÓN:

Los hábitos constantes hacen más eficiente el aprendizaje de programación. La práctica organizada permite avanzar con mayor seguridad y autonomía.

Explicación

Este prompt cumple con las cuatro reglas de sintaxis avanzada.

La **prioridad posicional** aparece desde el comienzo mediante la instrucción “**IMPORTANTE**” y la indicación de respetar exactamente las instrucciones.

El **aislamiento de variables** se consigue utilizando los delimitadores <<< >>> para separar claramente el contexto de las instrucciones.

La **especificidad** se observa en la cantidad exacta de hábitos, la cantidad de oraciones, el tono, la estructura y el formato solicitado.

Finalmente, se utiliza una **formulación positiva**, indicando las acciones que el modelo debe realizar, como “elaborá”, “incluí”, “usá”, “presentá” y “cerrá”.

CONCLUSIÓN GENERAL

A través de las diferentes actividades se pudo comprobar que la forma en que se redacta un prompt influye directamente sobre el comportamiento esperado de un modelo de lenguaje.

La temperatura permite orientar el modelo hacia respuestas más deterministas o creativas. Las restricciones de longitud permiten controlar la cantidad de información generada, mientras que las secuencias de parada permiten establecer un punto específico de finalización.

Las penalizaciones de frecuencia y presencia permiten modificar la repetición y la aparición de nuevos conceptos. Finalmente, una estructura avanzada que combine prioridad, aislamiento del contexto, especificidad y formulaciones positivas permite crear instrucciones más claras, organizadas y controlables.

De esta manera, la ingeniería de prompts permite mejorar la calidad y previsibilidad de las respuestas obtenidas de los modelos de lenguaje.

ENTREGA EN GITHUB

Una vez terminado el documento:

1. Guardar el trabajo en Word o Google Docs.
2. Insertar las **capturas reales de evidencia** correspondientes a cada actividad.
3. Exportar el documento a **PDF**.
4. Crear un **repositorio público en GitHub**.
5. Subir el PDF al repositorio.
6. Comprobar que el PDF sea visible y accesible.
7. Copiar el enlace del repositorio.
8. Entregar únicamente ese enlace, según las instrucciones de la actividad.